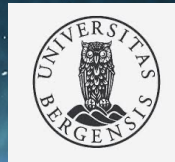


Distributed LLM Fine-Tuning & Inference on HPC Systems



Norwegian research infrastructure services

Hicham Agueny, PhD
Scientific Computing Group
Info.Tech. Department, UiB/NRIS



Day 2 —

Distributed Training & Optimized Inference

09:30 – 10:15 | Distributed Training Concepts

- DDP vs FSDP
- Communication overhead and scaling efficiency

Hands-On Session – Practical Multi-GPU Fine-Tuning & Inference

Multi-GPU Fine-Tuning (Alpaca dataset)

10:30 – 11:15 | Multi-GPU LoRA fine-tuning

11:15 – 12:00 | Multi-GPU QLoRA fine-tuning

Inference: Question-Answering & Summarization

13:00 – 13:30 | Introduction to the vLLM inference engine

13:30 – 14:15 | Single-GPU inference benchmarking (LoRA & QLoRA)

14:30 – 15:30 | Multi-GPU inference (LoRA & QLoRA)

15:30 – 16:00 | Wrap-up & Discussion



Learning Outcomes

- Scale up fine-tuning to multiple GPUs with LoRA & QLoRA.
- Perform distributed inference with vLLM.
- Monitor GPU usage and memory utilization & Profiling.



Challenge with modern LLMs for fine-tuning

During fine-tuning, you need to store:

1. **Weights**: The model parameters themselves
2. **Gradients**: Computed during backpropagation for weight updates
3. **Optimizer States**: Adam maintains two estimates per parameter (m, v)
4. **Activations**: Intermediate outputs for backpropagation

Ex. Kimi 2.5 1 trillion parameters

FP32 (4 bytes): at least $1 \text{ TB} \times 4 \times 4 = 16 \text{ TB}$

FP16 (2 bytes): at least $1 \text{ TB} \times 2 \times 4 = 8 \text{ TB}$

FP4 (1/2 bytes): at least $1 \text{ TB} \times 0.5 \times 4 = 2 \text{ TB}$

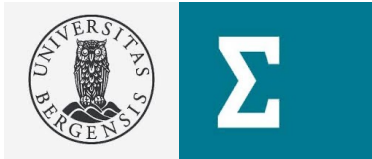


Model is too large to fit in a single GPU-memory

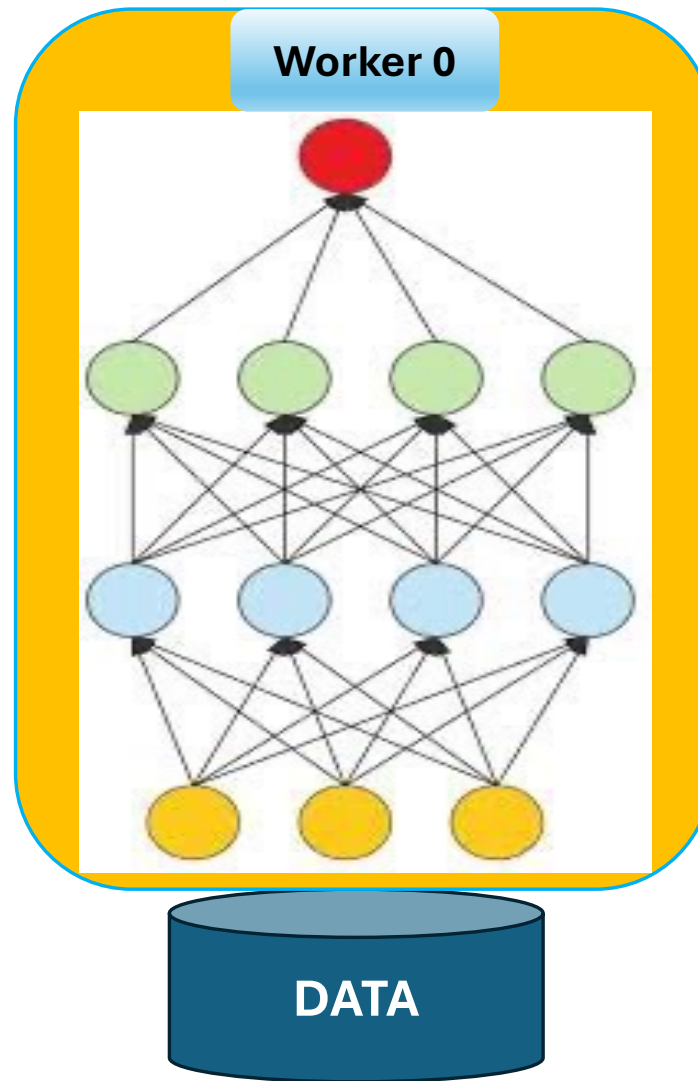
There is a need of a concept of parallelism to address the memory issue of large model

Concept of Distributed DNN Training

- **Model Parallelism**
- **Data Parallelism**
- **Concept FSDP – Fully Sharded Data Parallel**
- **Scalability**



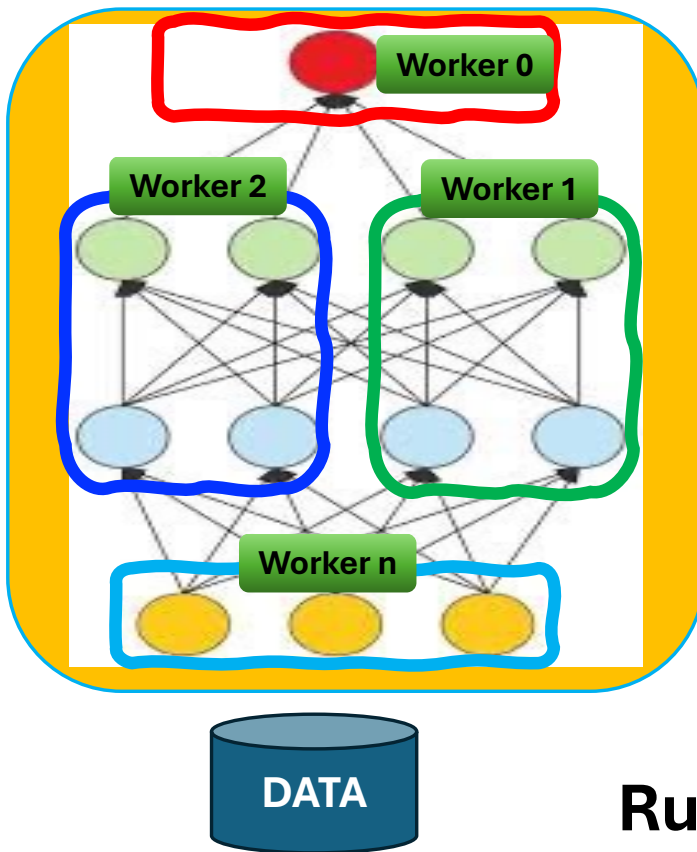
DNN Training on a single worker



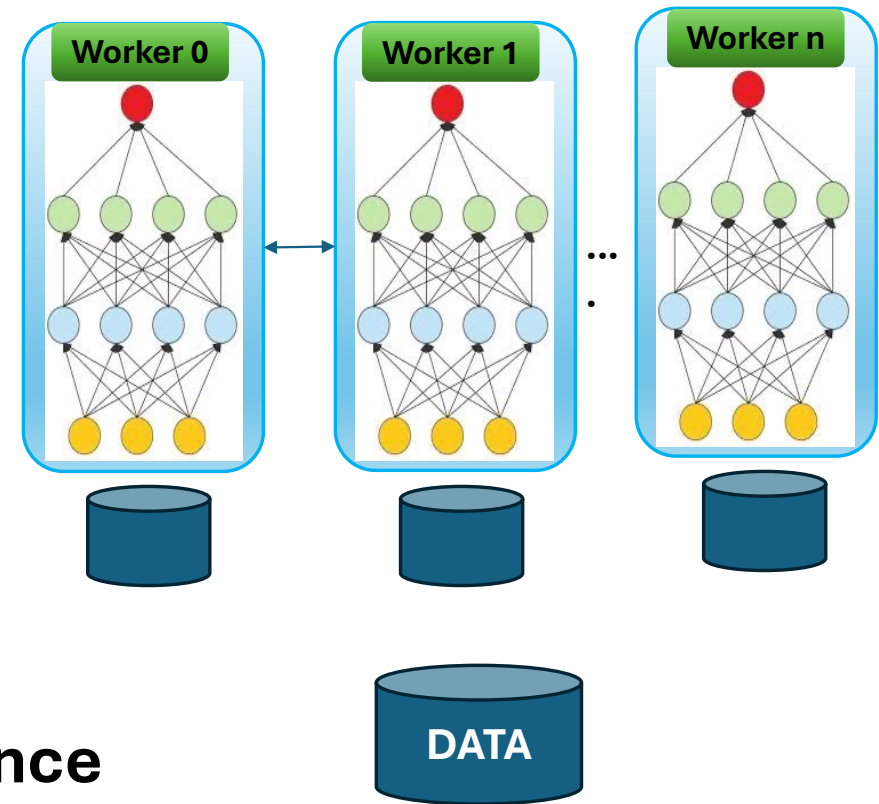
Distributed DNN Training: Parallelism Schemes

Parallelism in DNN: Training **large DNN models** or **large dataset** on **multiple Workers** in a shared or distributed environment.

Model Parallelism



Data Parallelism

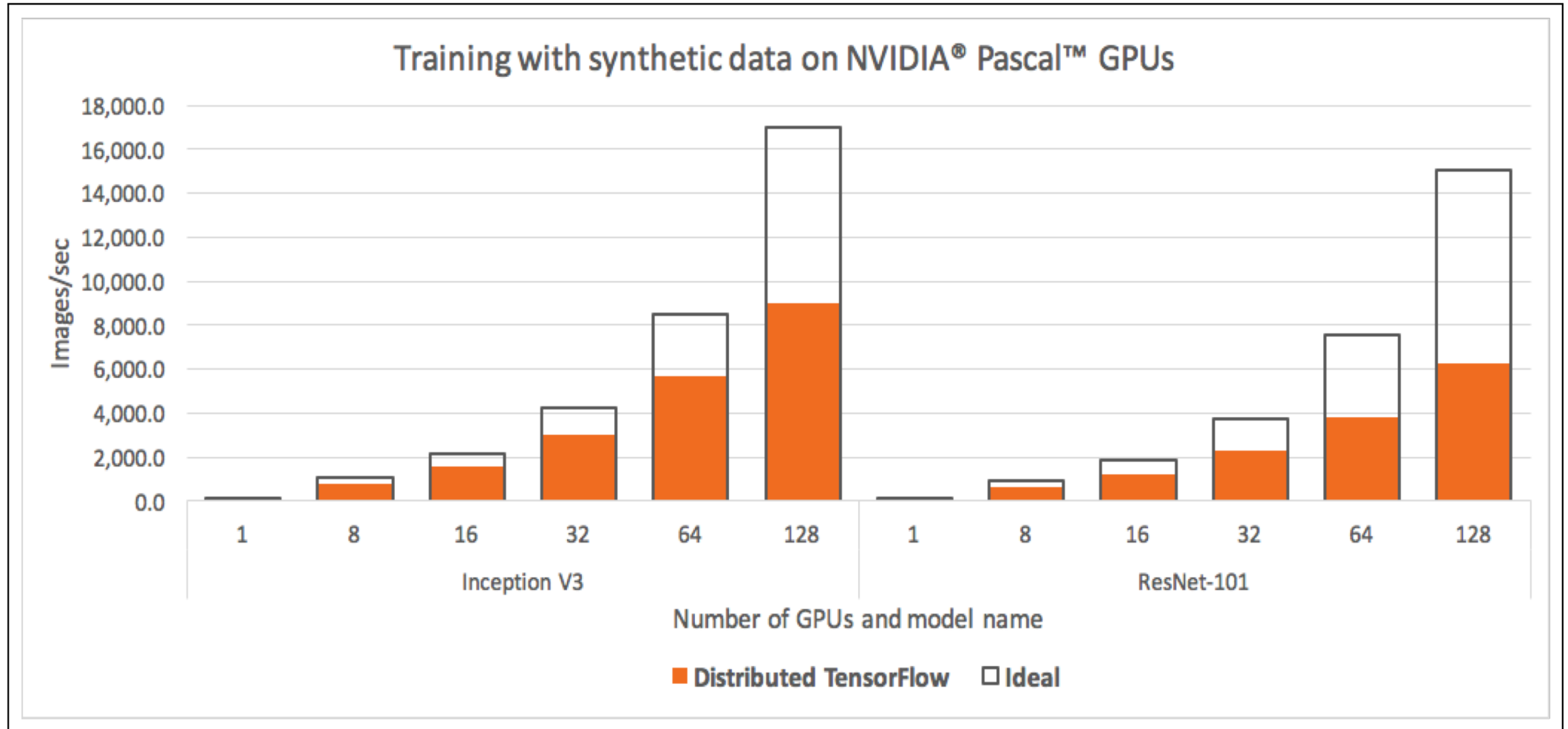


**Simplicity
&
Scalability
&**

Runtime Performance

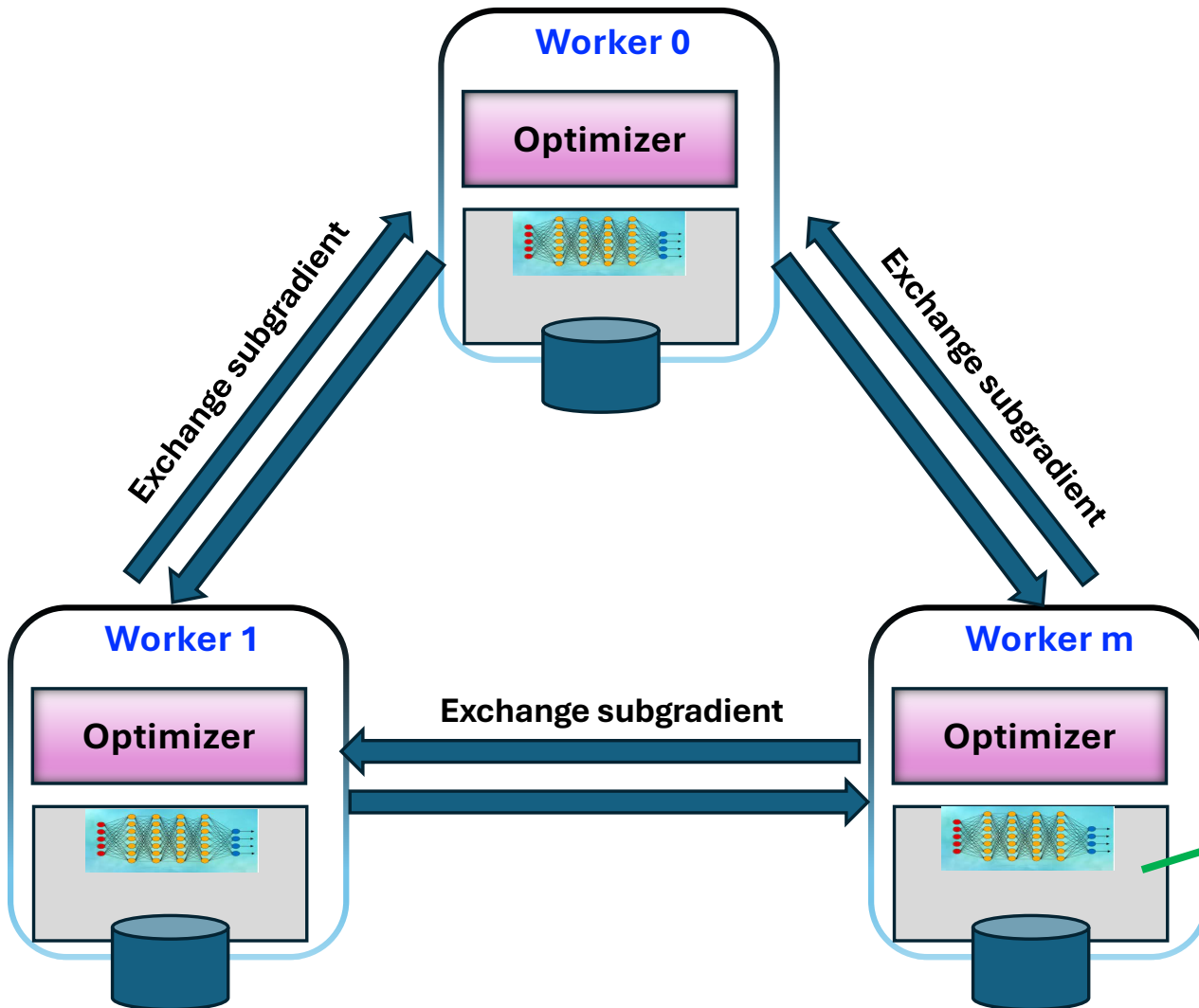
Distributed TensorFlow – Model parallelism

- Poor Scaling (communication overhead)



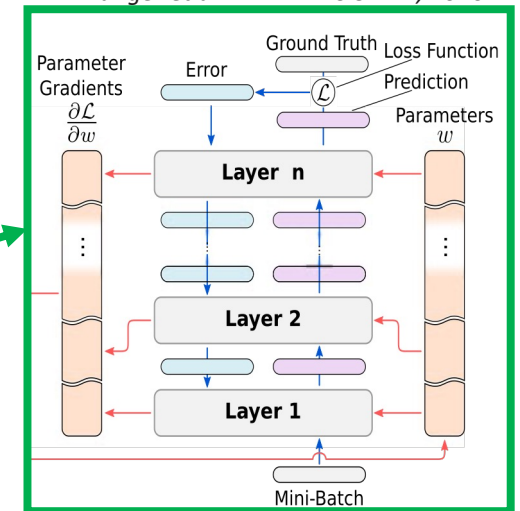
Alexander Sergeev, Mike Del Balso <https://arxiv.org/abs/1802.05799>

Decentralized Distributed DNN Training:



- Each worker computes (sub)gradient and exchange its value with the neighboring workers (forming a Ring)
- Each worker computes their own weights
- Each worker don't exchange weights with other workers

M. Langer et al. IEEE TPDS 31:12, 2020



Limitation of DDP – Distributed Data Parallel

- Each device holds a complete replica of parameters
- Model must be fully materialized on each GPU
- Initialization requires real tensor allocation in GPU memory
- Fails for very large models (cannot fit on a single GPU)

What's needed: a solution that achieves the performance of DDP
and addresses the issue of scalability of large models

One of the solution is FSDP – Fully Sharded Data Parallel

Concept of PyTorch FSDP - Fully Sharded Data Parallel

FSDP is a distributed training approach designed to train Large deep learning models that exceed the memory capacity of a single GPU.

Core idea: Sharding (split & distribute) model parameters, gradients, and optimizer states across available GPUs.

“**Sharding**” here means: **Splitting tensors into pieces and distribute those pieces across GPUs instead of replicating the full tensor on every GPU.**

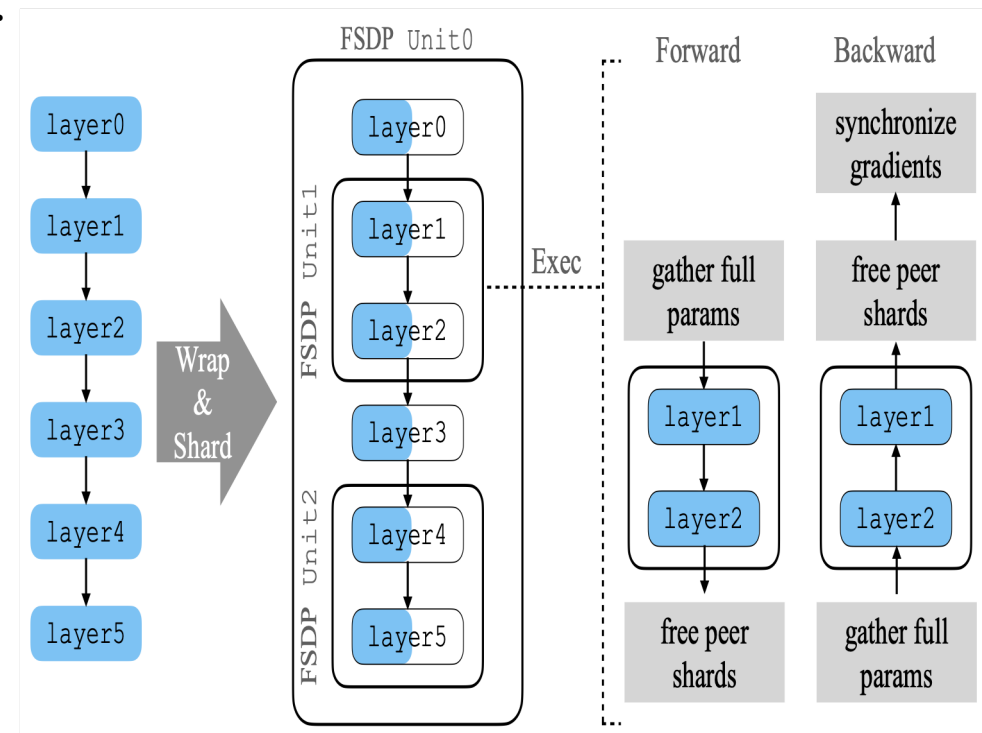
The Sharding Mechanism

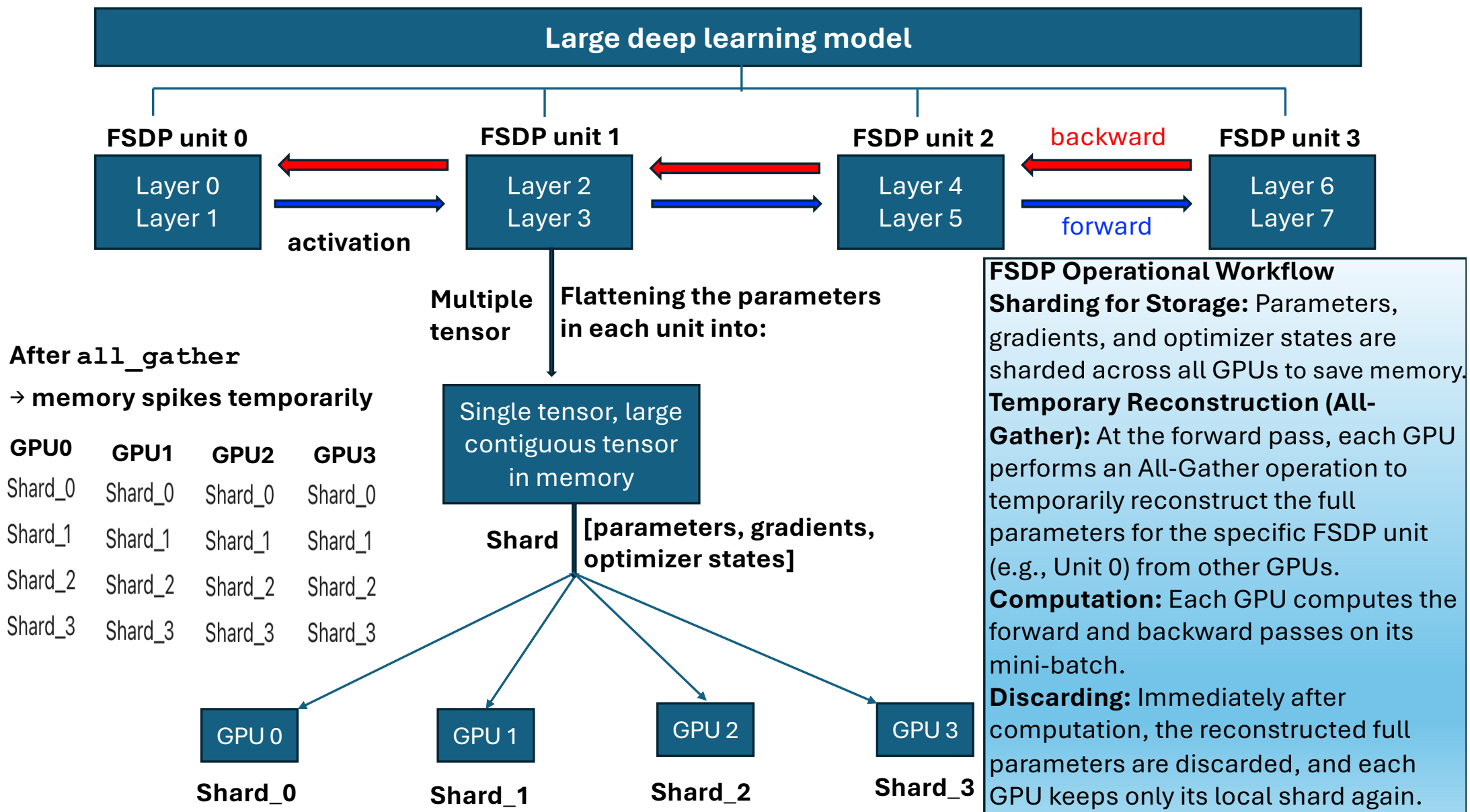
Decomposition: The model is broken down into smaller components called **FSDP units**.

Flattening & Sharding: Parameters within each unit are flattened and distributed across all available devices.

On-Demand Recovery: Parameters are temporarily "unsharded" (gathered) only when needed for computation. Peer shards are **immediately discarded** after use to minimize peak memory consumption.

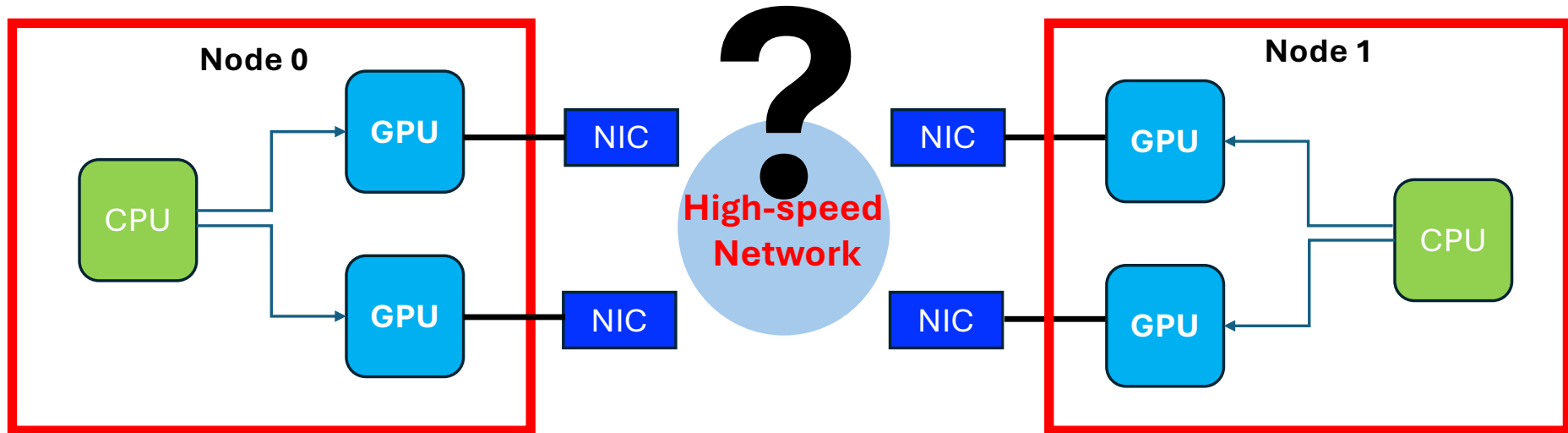
<https://arxiv.org/abs/2304.11277>





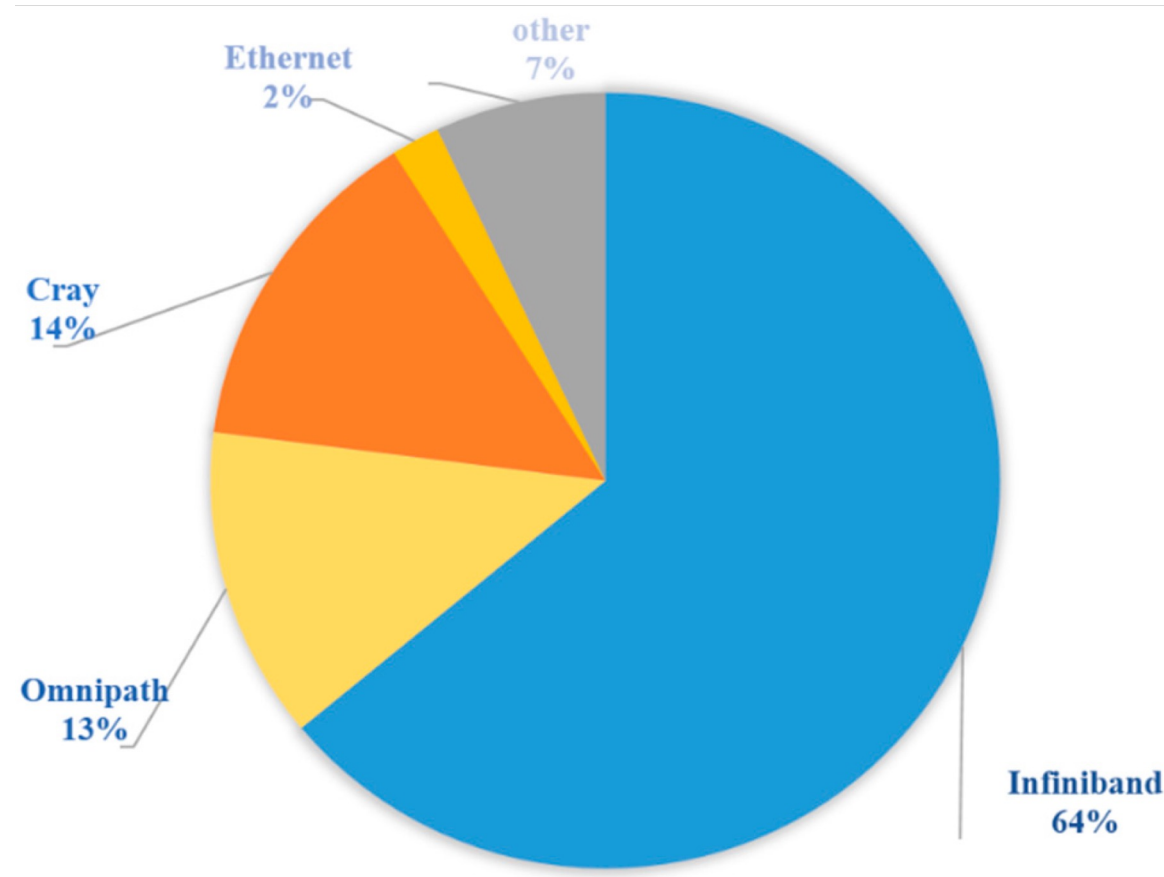
Communication

Network Interface Card - NIC



How does a compute node connect to thousands of other compute nodes in an HPC system ??

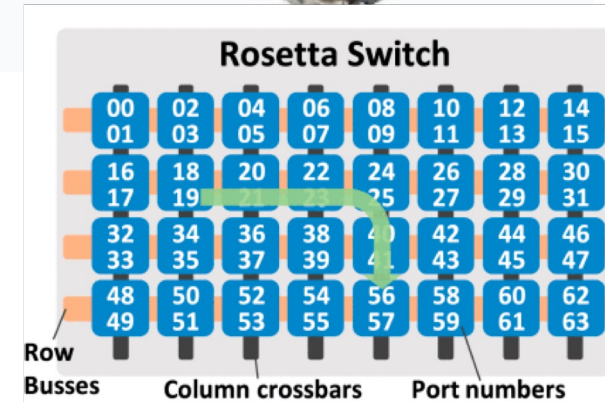
high-performance interconnects - HPIs



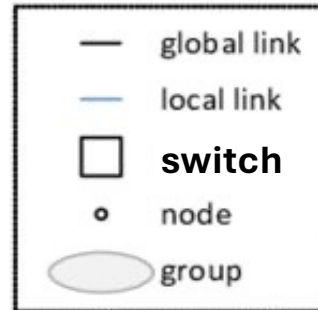
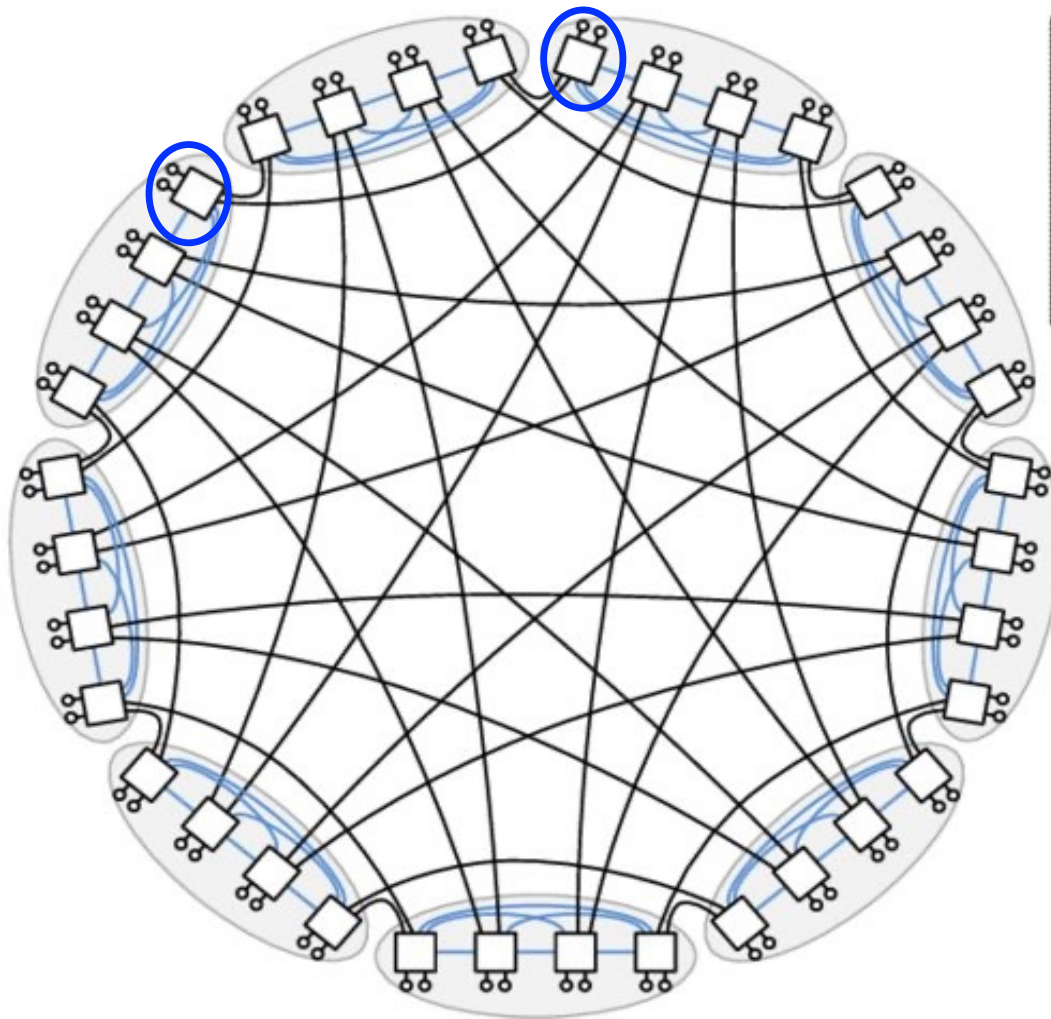
Distribution map of HPIs in the Top100 as of Nov 2021

HPE Slingshot interconnects

- **HPE Slingshot networks** are constructed from two components:
 - **Cassini** – a PCIe Gen4 Network Interface cards (NIC)
 - **Rosetta** – a 64-port **200 Gbps Ethernet switch** (Slingshot Switch)
- HPE Slingshot uses **Dragonfly topology** to ensure
 - High-bandwidth
 - Low latency
 - Scalability
 - Cost effectiveness



Dragonfly topology: Example of 72 end-points (compute nodes)



Dragonflies are organized as groups of switches.

Switches form a network that allows any node to reach any other node.

Ex. 9 groups

Each has 4 switches

Each switch connects 2 compute nodes

Total of $9 \times 4 \times 2 = 72$ end-points (compute nodes)

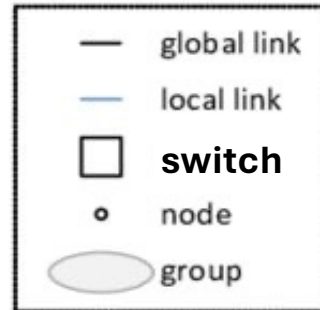
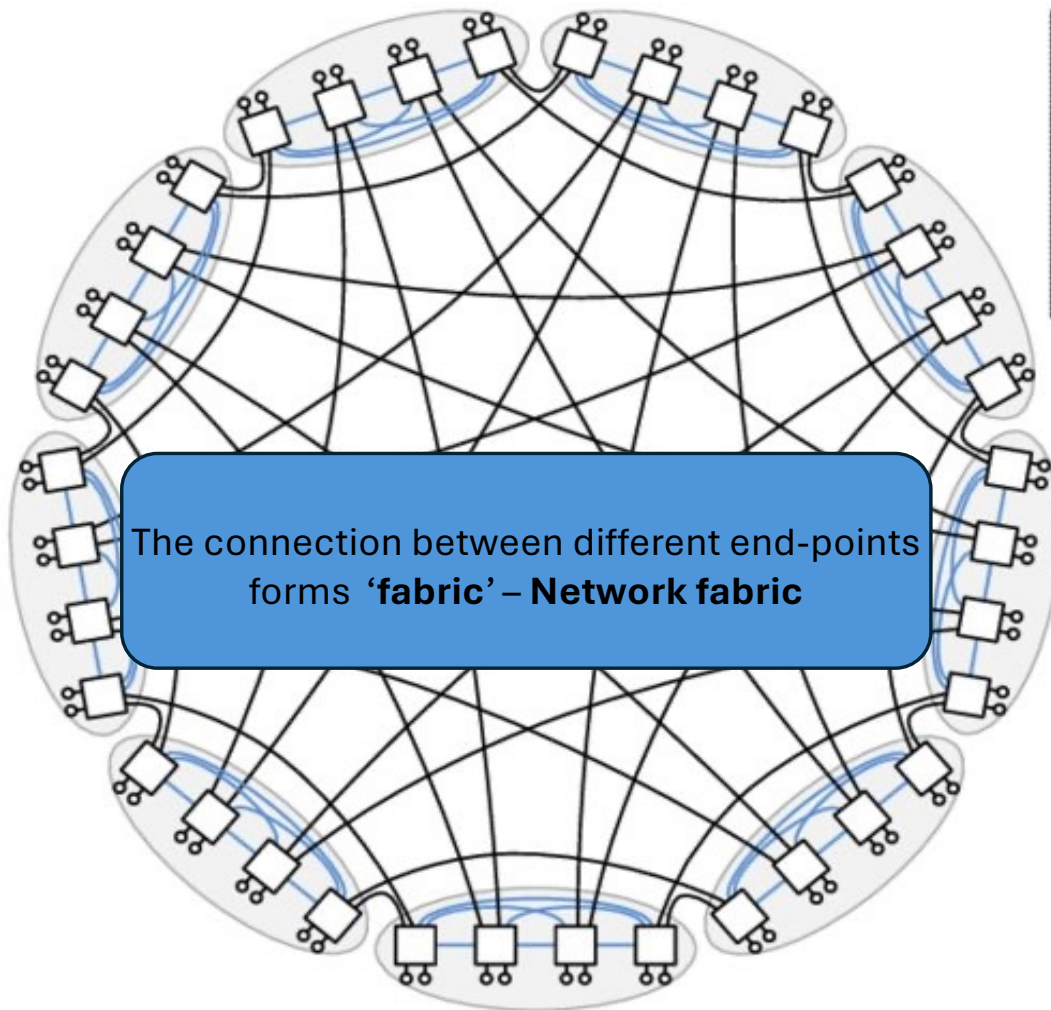
A compute node can communicate with any other node in at most 3 hops (steps)

1. Node A (Source) → Switch 1 (1st hop)

2. Switch 1 → Switch 2 (2nd hop)

3. Switch 2 → Node B (Destination)

Dragonfly topology: Example of 72 end-points (compute nodes)



Dragonflies are organized as groups of switches.

Switches form a network that allows any node to reach any other node.

Ex. 9 groups

Each has 4 switches

Each switch connects 2 compute nodes

Total of $9 \times 4 \times 2 = 72$ end-points (compute nodes)

A compute node can communicate with any other node in at most 3 hops (steps)

1. Node A (Source) → Switch 1 (1st hop)
2. Switch 1 → Switch 2 (2nd hop)
3. Switch 2 → Switch 3 (3rd hop)
4. Switch 3 → Node B (Destination)

Scalability

Speedup:

$$S = \frac{T_{single}}{T_{multi}}$$

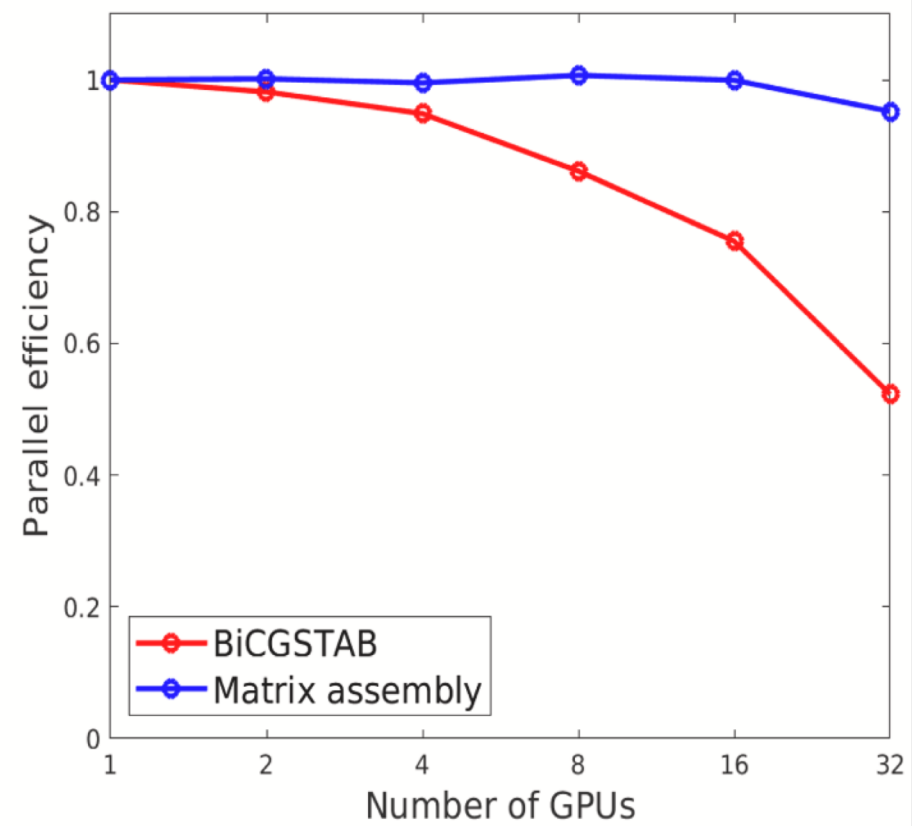
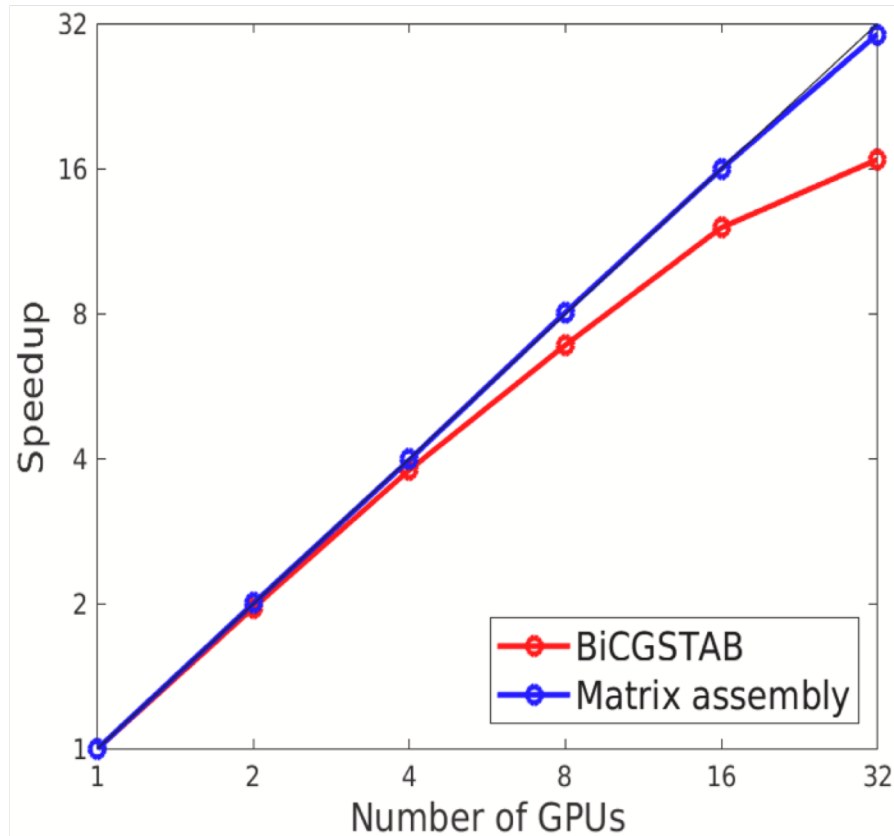
T_{single} : execution time on a single GPU

T_{multi} : execution time on multiple GPUs

N_p : number of GPUs

Parallel efficiency

$$P_e = \frac{1}{N_p} \frac{T_{single}}{T_{multi}}$$



Scalability - benchmarking

- Include warm-up steps
 - Run 2 initial steps as warm-up
 - The first step involves:
 - Memory allocation, Kernel initialization, Compilation (slow down the timing)
- Standardize scalability tests
 - Measure performance over a fixed window of 5–10 training steps (post warm-up)
 - Compute the average execution time per step for consistency.
- Monitor throughput
 - Track tokens per second per GPU
 - Use as a metric to assess scaling efficiency



Hands-on:

Fine-Tuning on multiple GPUs

Dataset: Alpaca dataset

Question Answering tasks

Hands-on: Fine-Tuning on multi GPUs

Dataset: Alpaca dataset: Question Answering (QA) tasks

The dataset is stored in this path:

`/cluster/work/projects/nn9970k/$USER/llm-hpc-course/shared/datasets/GPT-4-LLM/alpaca_gpt4_data.json`

The config file is available in this path:

`/cluster/work/projects/nn9970k/$USER/llm-hpc-course/configs/lora/llama3_1_8B_lora_multi_device_alpaca_gpt4.yaml`

LoRA slurm job:

`/cluster/work/projects/nn9970k/$USER/llm-hpc-course/day2_multi_gpu/fine-tuning/lora_slurm/job_multiGPU_LoRA.sh`

`/cluster/work/projects/nn9970k/hicham/llm-hpc-course/day2_multi_gpu/fine-tuning/qlora_slurm/job_multiGPU_QLoRA.sh`

- **Hands-On Workflow**

Instructions for Fine-tuning on multiple GPUs: [day2_multi_gpu/README.md](#)

0. Explore SLURM job scripts.

1. Review configuration files for LoRA Fine-tuning.

2. Submit jobs and monitor GPU usage.

3. Compare metrics: single GPU vs multiple GPUs

4. Repeat the experiment with QLoRA



Overview of vLLM inference engine

Efficient Memory Management for Large Language Model Serving with *PagedAttention*

Woosuk Kwon^{1,*} Zhuohan Li^{1,*} Siyuan Zhuang¹ Ying Sheng^{1,2} Lianmin Zheng¹ Cody Hao Yu³
Joseph E. Gonzalez¹ Hao Zhang⁴ Ion Stoica¹

¹UC Berkeley ²Stanford University ³Independent Researcher ⁴UC San Diego

What is it? vLLM is an Open-source library for Large Language Model (LLM) **inference** and **serving**.

Origin: Developed initially by researchers at UC Berkeley.

The Problem: LLM inference is traditionally **memory-bound**, not compute-bound.

Standard serving frameworks struggle with memory fragmentation, leading to small batch sizes and wasted GPU resources (static memory allocation).

The Solution: vLLM improves throughput by managing memory in non-contiguous blocks, eliminating fragmentation.

vLLM architecture

Scheduler:

To coordinate the execution of distributed GPU workers.

KV cache manager:

Manages the physical KV cache memory on the GPU workers through the instructions sent by the centralized scheduler.

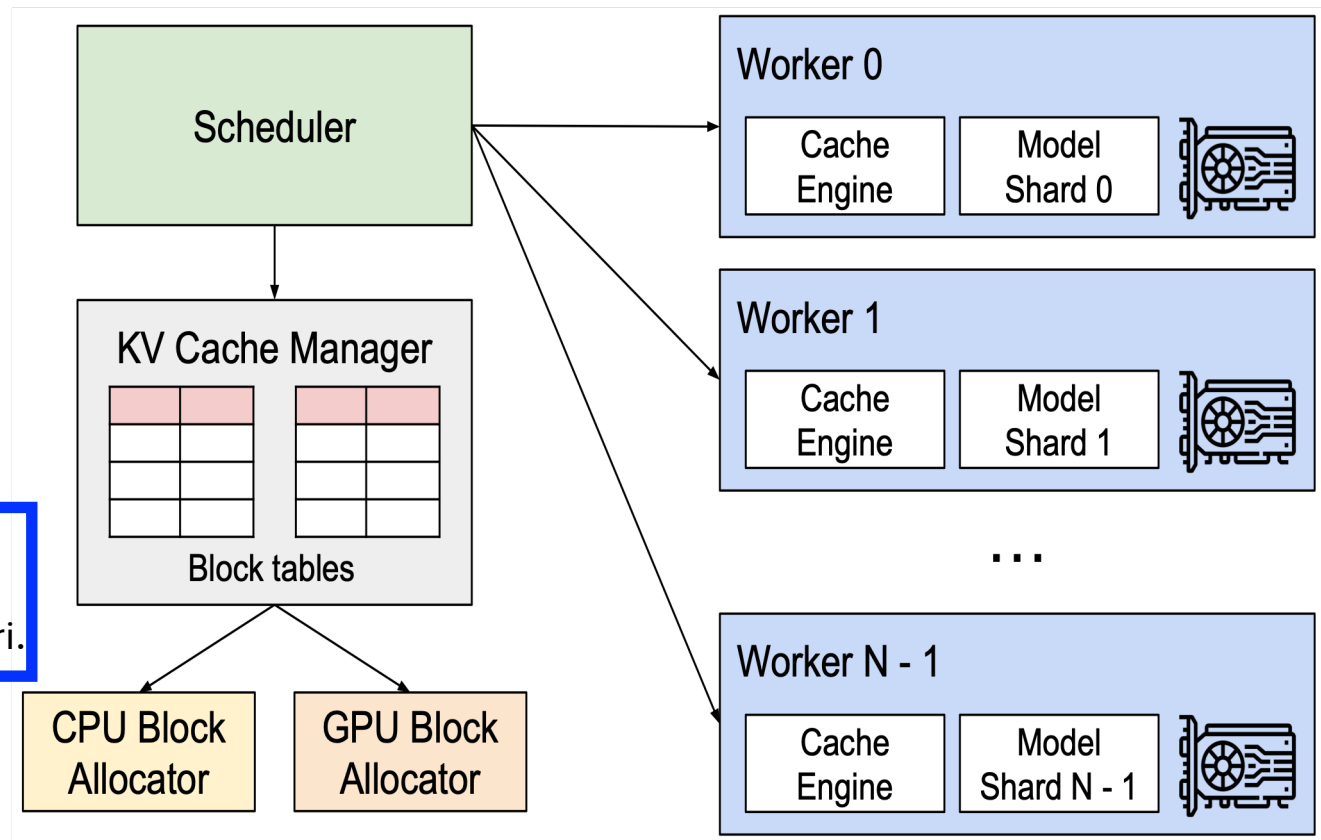
KV cache dynamically grows and shrinks over time as the model generates new tokens, and its lifetime and length are not known a priori.

PagedAttention algorithm

Divides the request's KV cache into blocks, each of which can contain the attention keys and values of a fixed number of tokens.

Key feature: "Continuous batching" - grouping a bunch of different user requests together to process them simultaneously and save compute time.

Quantization: supports heavily compressed models to further reduce the memory footprint.



<https://arxiv.org/pdf/2309.06180>

Distributed Inference Strategy

1. Single GPU (No Distribution)

- Use when the model fits in one GPU
- Simplest setup, lowest overhead

2. Single Node, Multi-GPU (Tensor Parallelism)

- Use when the model doesn't fit in one GPU but fits in one machine
- Split model across GPUs within the same node
- *Tensor parallel size = number of GPUs (e.g., 4 GPUs $\rightarrow$ TP = 4)*

3. Multi-Node, Multi-GPU (Tensor + Pipeline Parallelism)

- Use when the model exceeds a single node's capacity
- Combine:
 - Tensor parallelism (within each node)
 - Pipeline parallelism (across nodes)
- *Example: 16 GPUs across 2 nodes $\rightarrow$ TP = 8, PP = 2*

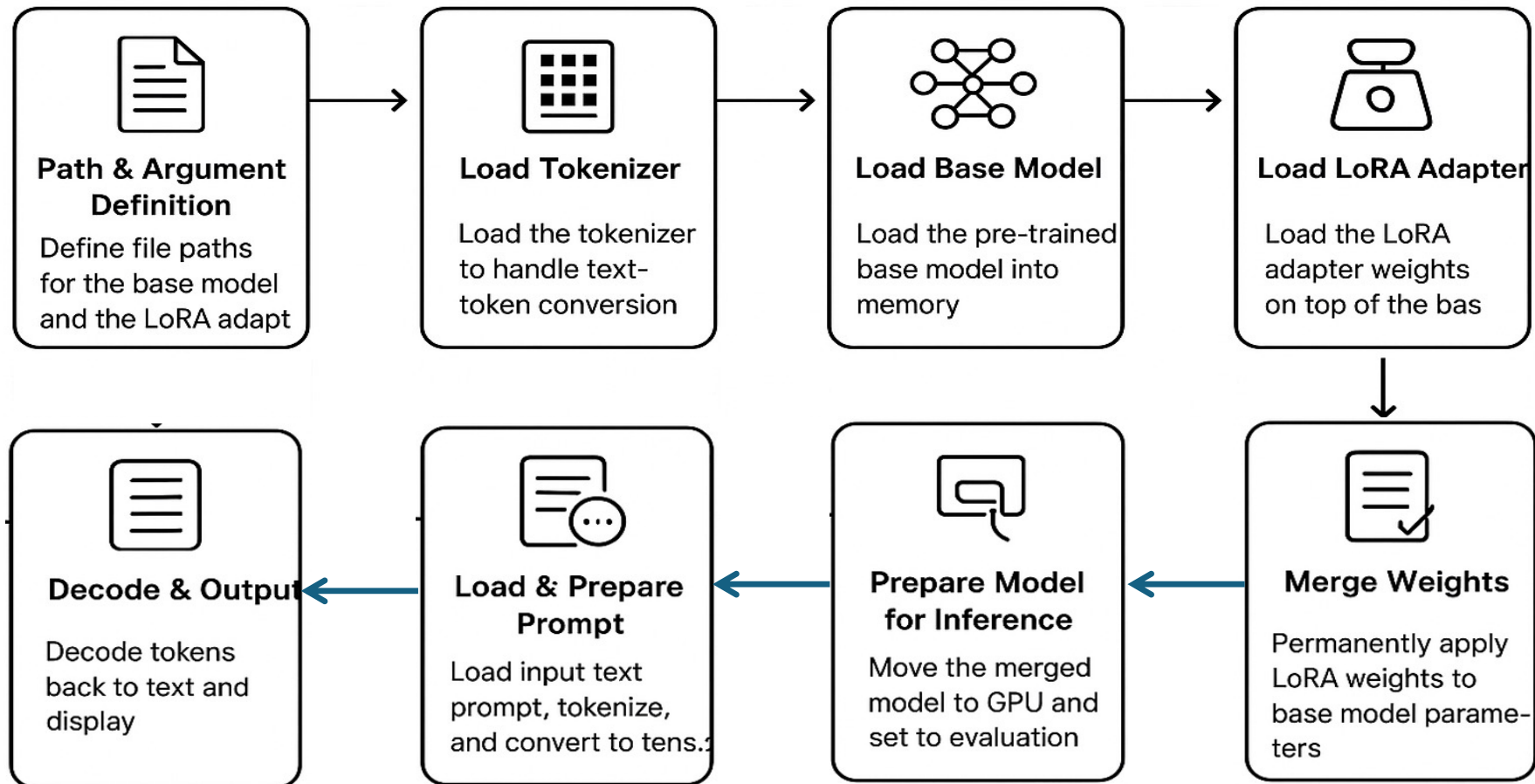
Scale from **single GPU** $\rightarrow$ **multi-GPU (single node)** $\rightarrow$ **multi-node** as model size exceeds hardware limits.

https://docs.vllm.ai/en/v0.8.1/serving/distributed_serving.html

Hands-on:

vLLM inference: single GPU and multi GPUs

Structure of the Inference process



Single GPU inference

Task: Summarization

Run an interactive script

`./srun_interactive_1gpu`

For LoRA inference:

`./vllm_lora.sh`

For QLoRA inference

`./vllm_qlora`

Experiment with different prompts

Codes are taken from here (adapted):

Single GPU LoRA/QLoRA inference:

https://docs.vllm.ai/en/v0.11.0/examples/offline_inference/lora_with_quantization_inference.html



HichamAgueny update slurm job

Name	Last commit message
 ..	
 prompt_QA.json	add interactive example
 srun_interactive_1gpu	update slurm job
 vllm_lora.sh	update project nbr
 vllm_qlora.sh	update project nbr

Distributed inference

Task: QA

For distributed LoRA inference:
sbatch slurm_vllm_distributed.sh

Experiment with different prompts

Codes are taken from here (adapted):

Distributed inference:
https://docs.vllm.ai/en/v0.11.0/examples/offline_inference/data_parallel.html



HichamAgueny update slurm job

Name	Last commit message
 ..	
 prompt_QA.json	add features in day2_multi_gpu
 slurm_vllm_distributed.sh	update slurm job
 slurm_vllm_lora.sh	update slurm job
 slurm_vllm_qlora.sh	update slurm job



Wrap up & Discussion

Throughput per-GPU:

Model: Llama 3.2-1B

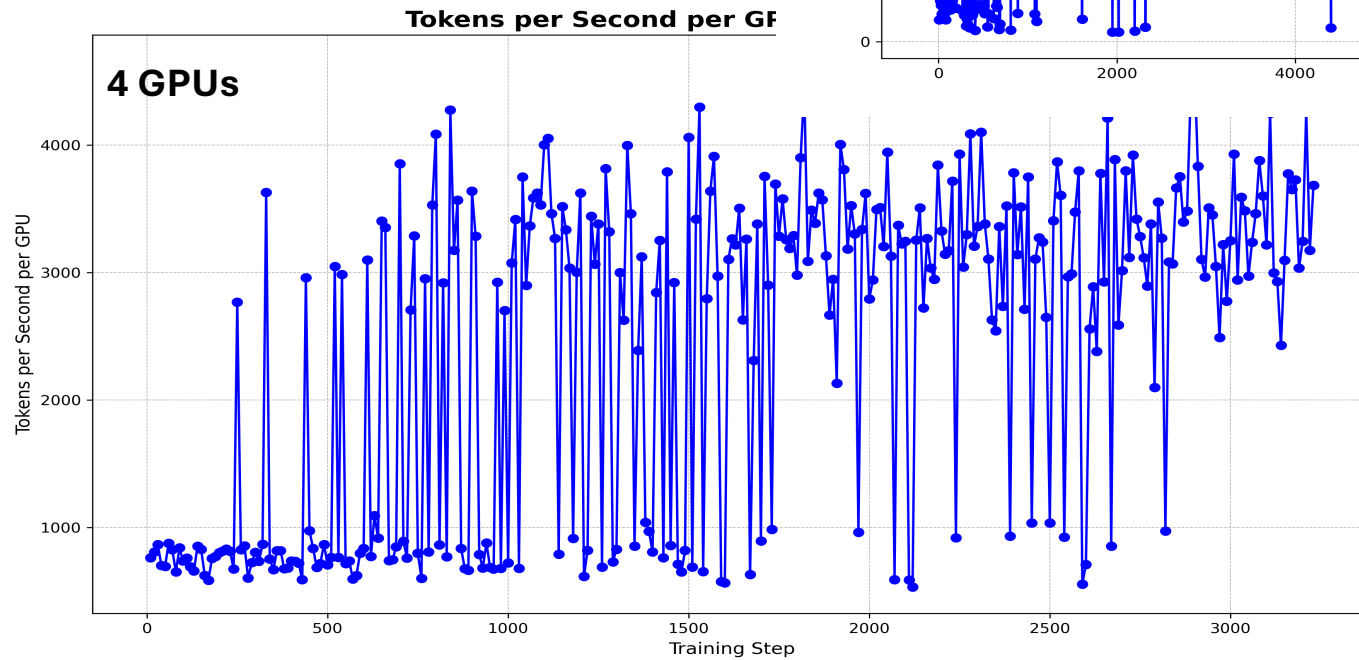
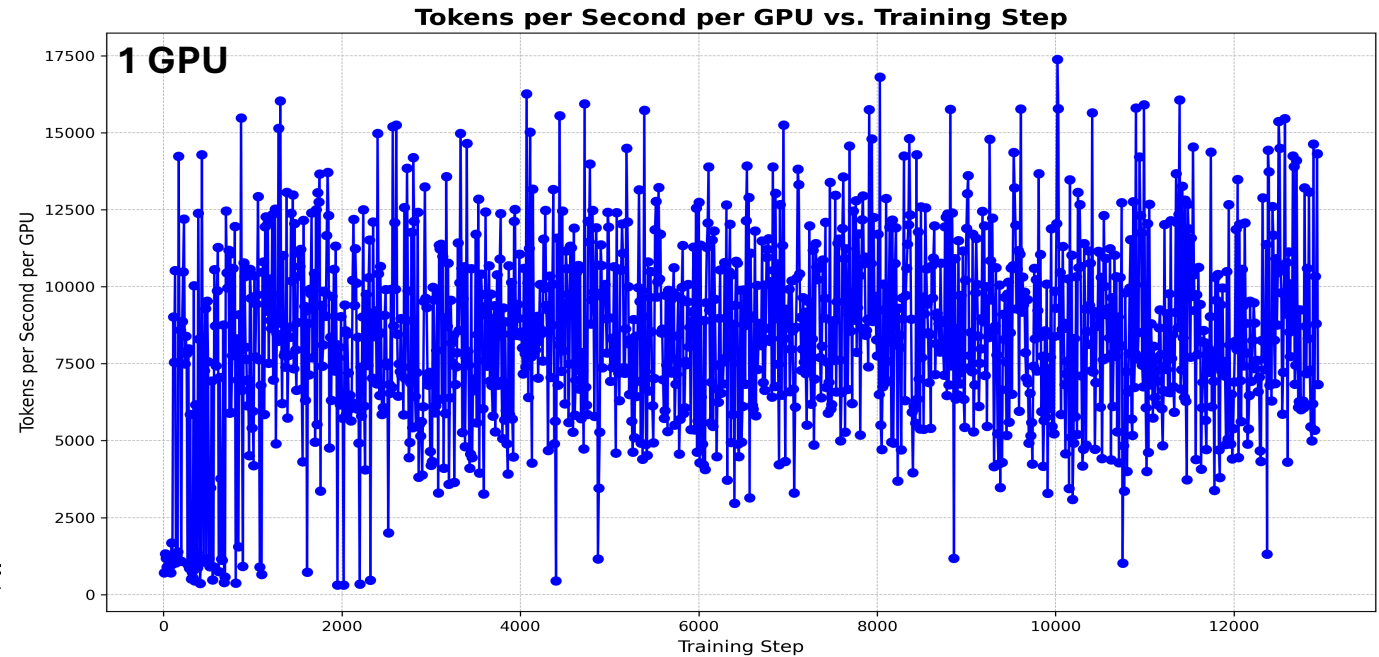
1 GPU: up to **13000** Tokens per sec per GPU

4 GPUs: up to **4000** Token per sec per GPU

Total system throughput : $4 \times 4000 = 16000$

Runtime (1 GPU): 0.12 s/training step

Runtime (4 GPUs): 0.10 s/training step



Parallel efficiency of $\approx 30\%$

1B model, doesn't benefit from multi-GPU training

A single GPU can easily fit the model in memory and process batches efficiently.

Additional overheads in multiGPU setup

Throughput per-GPU:

Model: **Llama 3.2-11B**

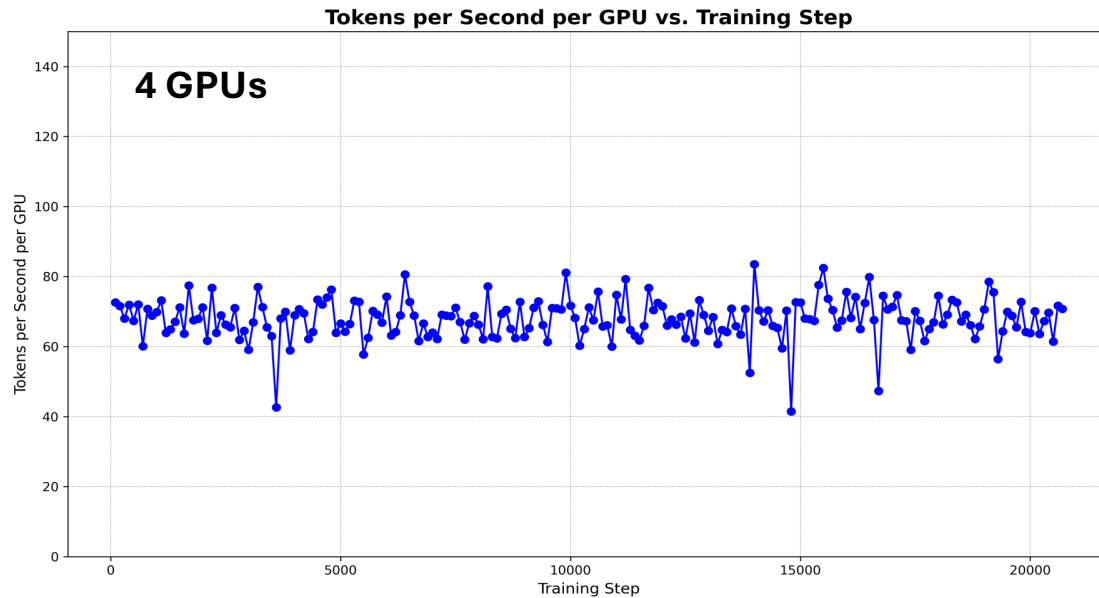
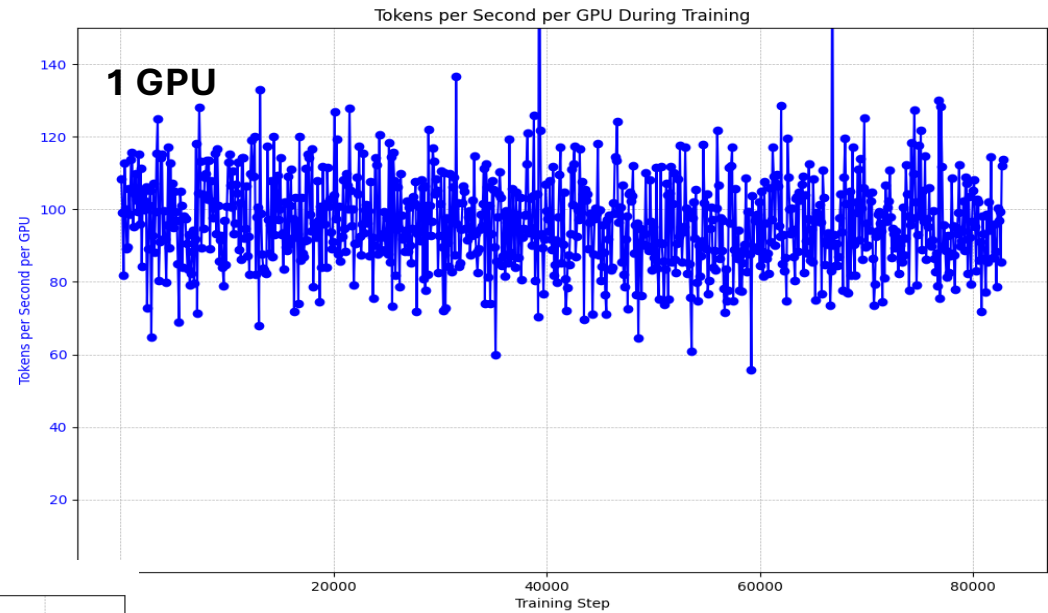
1GPU: up to **110** Tokens per sec per GPU

4 GPUs: up to **70** Token per sec per GPU

Total system throughput : $4 \times 70 = 280$

Runtime (1GPU): 1.1 second/training step (**25.20 h**)

Runtime (4 GPUs): 0.4 s/training step (**9.04 h**)



Scaling efficiency:

Parallel efficiency of $\approx 70\%$ on 4 GPUs

Using **4 GPUs** reduced the training time by a **factor of 2.8**

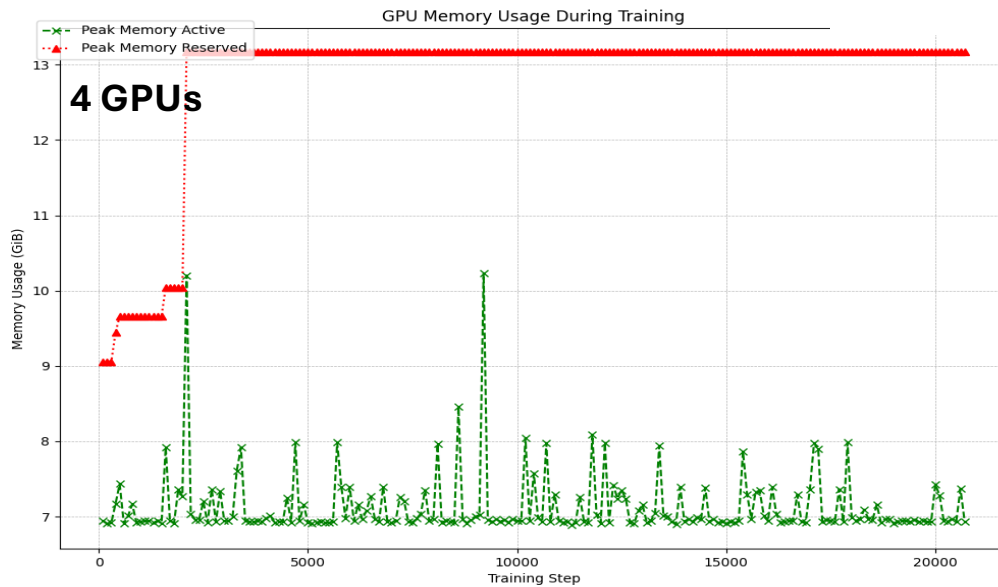
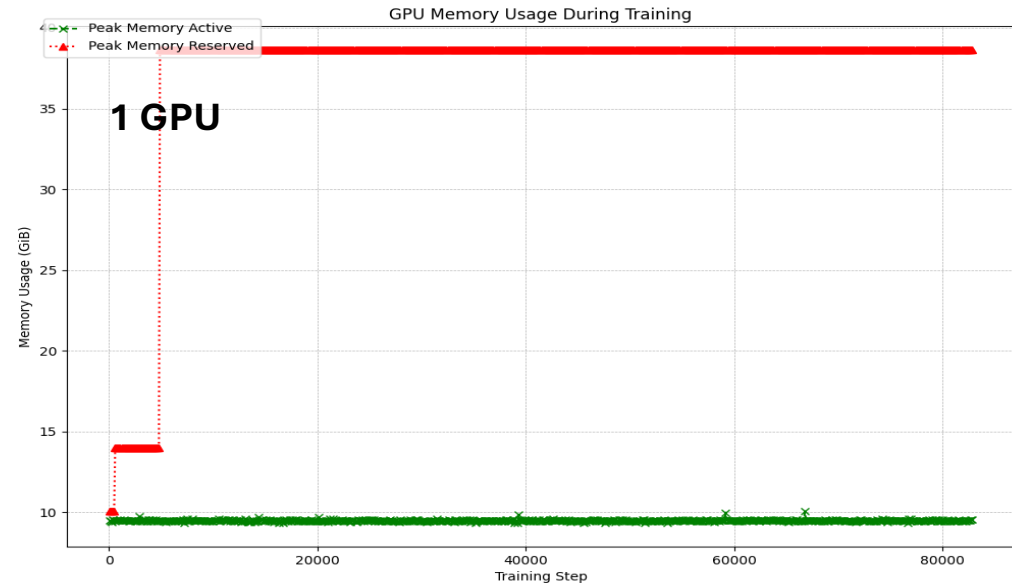
For much larger models, where the model itself cannot fit on a single GPU.....

Memory per-GPU:

Model: **Llama 3.2-11B**

1 GPU: Peak mem active is **9-10 GiB**
peak mem reserved is up to **38.6 GiB**

4 GPUs: Peak mem active is **7-8 GiB**
Peak mem reserved is up to **13.2 GiB**



Moving from **1 GPU** to **4 GPUs** **reduces** the **peak reserved memory** by a **factor of 3**.
(but slightly lowering the peak active memory)

Efficient distribution of memory across the GPUs.

References

- **LoRA:** <https://arxiv.org/pdf/2106.09685>
- **LoRA Dropout:** <https://arxiv.org/pdf/2404.09610>
- **QLoRA:** <https://arxiv.org/pdf/2305.14314>
- **PEFT:** <https://arxiv.org/abs/2312.12148>
- **PyTorch FSDP:** <https://arxiv.org/pdf/2304.11277>
- **ZeRO:** <https://arxiv.org/abs/1910.02054>
- **Ultimate Guide to Fine-Tuning LLMs:** <https://arxiv.org/pdf/2408.13296>
- **Merging model:** https://huggingface.co/docs/peft/en/developer_guides/lora#weight-decomposed-low-rank-adaptation-dora
- **Transformer:** <https://arxiv.org/pdf/1706.03762>
- **Fastformer:** <https://arxiv.org/pdf/2108.09084>
- **vLLM-PagedAttention:** <https://arxiv.org/pdf/2309.06180>
- **vLLM examples:** https://docs.vllm.ai/en/v0.11.0/examples/offline_inference/data_parallel.html
- **PyTorch container:** <https://docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel-24-09.html>
- **GitHub repo LLM-course:** <https://github.com/HichamAgueny/llm-hpc-course>